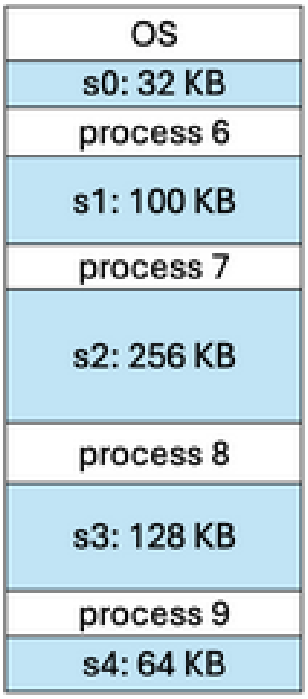
Assignment 2: Threads, Scheduling, Synchronization

Suhaib Abdi Muhummed, muhummed@kth.se
Abdulaziz Hassan Farah, ahfarah@kth.se
Daniel Sherif, dsherif@kth.se

Question 1: memory allocation and fragmentation

Assume free partitions are stored in order of {S0, S1, S2, S3 S4} and their sizes are shown in the diagram below. Processes with allocated partitions are in white blocks, and free partitions are in blue blocks.



Given a list of processes with memory allocation request in order of {P0, 115 KB} {P1, 500 KB} {P2, 256 KB} {P3, 31 KB}, show process allocation for each of the following algorithms. If a process cannot be allocated with memory, indicate it as {P0, X}.

1. how would the first-fit algorithms assign memory partitions.
2. how would the best-fit algorithms assign memory partitions.
3. how would the worst-fit algorithms assign memory partitions.
4. Propose your own metric for quantifying the memory fragmentation. Calculate this metric for each allocation algorithm. Explain how your metric can support make better memory allocation decisions.

Answer:

1. **First-Fit**

   We assign processes in the order {P0, P1, P2, P3} to the first free partition that is large enough.

   $$\{P_0, S_2\}, \qquad \{P_1, X\}, \qquad \{P_2, X\}, \qquad \{P_3, S_0\}$$

   Unused space:

   $$P_0 \to S_2 : 256 - 115 = 141$$

   $$P_3 \to S_0 : 32 - 31 = 1$$

   $$\text{Total unused space} = 141 + 1 = 142 \text{ KB}$$

   Unused blocks:

   $$\{S_1, S_3, S_4\}$$

2. **Best-Fit**

   Assignments:

   $$\{P_0, S_3\}, \qquad \{P_1, X\}, \qquad \{P_2, S_2\}, \qquad \{P_3, S_0\}$$

   Unused space:

   $$P_0 \to S_3 : 128 - 115 = 13$$

   $$P_2 \to S_2 : 256 - 256 = 0$$

   $$P_3 \to S_0 : 32 - 31 = 1$$

   $$\text{Total unused space} = 13 + 0 + 1 = 14 \text{ KB}$$

   Unused blocks:

   $$\{S_1, S_4\}$$

3. **Worst-Fit**

   Assignments:

$$\{P_0, S_2\}, \qquad \{P_1, X\}, \qquad \{P_2, X\}, \qquad \{P_3, S_3\}$$

   Unused space:

$$P_0 \to S_2 : 256 - 115 = 141$$

$$P_3 \to S_3 : 128 - 31 = 97$$

$$\text{Total unused space} = 141 + 97 = 238 \text{ KB}$$

   Unused blocks:

$$\{S_0, S_1, S_4\}$$

4. **Fragmentation Metric**

   We define a custom fragmentation metric as:

$$\text{Metric} = \frac{\text{Total unused space}}{\text{Number of unused blocks}}$$

   This metric measures how much unusable space exists per free block.

$$\text{Metric}_{FF} = \frac{142}{3}, \qquad \text{Metric}_{BF} = \frac{14}{2}, \qquad \text{Metric}_{WF} = \frac{238}{3}$$

Question 2: Address space and Pages

Assume the logical address space consists of 1024 pages with a page size of 4 KB. The physical memory consists of 64 frames.

1. Illustrate your process of determining the number of bits needed in the logical address.
2. Illustrate your process of determining the number of bits in the physical address.
3. Do you observe a mismatched number of pages and frames? Explain how it is possible to support it.

Assume the logical address space consists of 1024 pages with a page size of 64 KB. The physical memory consists of 1024 frames. Calculate the page numbers and offsets for each of the following logical addresses (they are in decimal numbers).

- 189150
- 43205425

Answer:

1. The logical address space consists of 1024 pages, and the page size is 4 KB.

   $$1 \text{ KB} = 1024 \text{ bytes} \Rightarrow 4 \text{ KB} = 4096 = 2^{12}$$

   Therefore, the number of offset bits is:

   $$d = \log_2(4096) = 12$$

   Since the system has 1024 pages:

   $$1024 = 2^{10} \Rightarrow p = 10$$

   Thus, the logical address requires:

   $$p + d = 10 + 12 = 22 \text{ bits}$$

2. Physical memory consists of 64 frames:

   $$64 = 2^6 \Rightarrow \text{frame number bits} = 6$$

   The offset size must match the logical address (same page size):

   $$d = 12$$

   Therefore, the physical address requires:

   $$6 + 12 = 18 \text{ bits}$$

3. Yes, there is a mismatch: the logical address space contains 1024 pages, but physical memory contains only 64 frames. This is completely valid in a virtual memory system. The page table maps each logical page either to a physical frame or to a location on disk. Only a subset of pages must be in physical memory at any given time. As long as the offset bits remain the same in both logical and physical

addresses (because page size = frame size), the CPU can always access the correct byte within a page or frame after translation.

Second scenario:

Logical address space: 1024 pages Page size: 64 KB Physical memory: 1024 frames

$$64 \text{ KB} = 64 \times 1024 = 65536 = 2^{16} \Rightarrow d = 16$$

$$1024 = 2^{10} \Rightarrow p = 10$$

Physical address frame bits:

$$1024 = 2^{10}$$

Logical address 189150:

Convert the address to its 26-bit binary form (10 page bits + 16 offset bits):

$$189150_{10} = 00\,0010\,1100\,0010\,1101\,1110_2$$

We split the address as follows:

$$\underbrace{0000000010}_{\text{page} = 2} \quad \underbrace{1100001011011110}_{\text{offset bits}}$$

The offset consists of the 16 least significant bits:

$$1100001011011110_2 = 58078_{10}$$

Logical address 43205425:

Binary representation of the 26-bit logical address:

$$43205425_{10} = 10\,1001\,0011\,0100\,0010\,0001_2$$

Splitting into page (10 MSB) and offset (16 LSB):

$$\underbrace{1010010011}_{\text{page} = 659} \quad \underbrace{0100001000010001}_{\text{offset bits}}$$

The offset value is obtained from the 16 least significant bits:

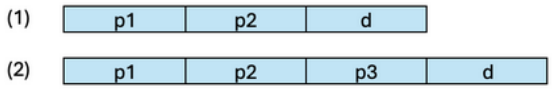
$$0100001000010001_2 = 17201_{10}$$

Question 3: Page Tables

A system uses 128-KB page size and one level paging. Assume we use a 64-bit logical address, and have the following address references: 650003, 42096, and 216202 (they are in decimal numbers). Please answer the follow questions.

1. What are the page numbers and offsets for the three address references?
2. If we use two-level TLB, TLB hit ratio is 99% in level 1 and 80% in level 2. TLB hit latency is 10 nanoseconds in level 1 and 30 nanoseconds in level 2. Memory access takes 200 nanoseconds. Please illustrate how TLB could help with the performance in your own words.

Now assume we use multi-level page tables as illustrated in (1) and (2). (1) uses 32-bit logical addresses and (2) uses 64-bit logical addresses. We use a page size of 4 KB and assume each inner page tables fit in one frame. Each page table entry takes 4B.



3. Please illustrate your process of determining the value of p1, p2, and d for those three memory references in (1) two-level page table.
4. Please illustrate your process of determining the value of p1, p2, p3, and d for those three memory references in (2) three-level page table

Answer:

1. Page Calculations

Page size = 128 KB = 128 × 1024 = 131 072 bytes

Page offset is $\log_2(131072) = 17$

Page number is 64 - 17 = 47 - where m is the systems total bits (64)

$$\text{Page number} = \left\lfloor \frac{\text{address}}{131072} \right\rfloor$$

$$\text{Offset} = \text{address mod } 131072 = \text{address} - (\text{page number} \times 131072)$$

Address 650,003

131,072 × 4 = 524,288

page number = $\lfloor 650,003 \div 131,072 \rfloor = 4$

offset = 650,003 - 524,288 = 125,715

Address 42,096

131,072 × 0 = 0

page number = $\lfloor 42,096 \div 131,072 \rfloor = 0$

offset = 42,096 - 0 = 42,096

Address 216,202

131,072 × 1 = 131,072

page number = $\lfloor 216,202 \div 131,072 \rfloor = 1$

offset = 216,202 - 131,072 = 85,130

---

2. TLB Effective Access Time

L1 TLB hit ratio = 99% (0.99).

L1 hit latency = 10 ns.

L2 TLB hit ratio (given for L1 misses) = 80% (0.8).

L2 hit latency = 30 ns.

Memory access time = 200 ns.

$$P(L1 \text{ Hit}) = 0.99$$

$$P(L1 \text{ miss} \wedge L2 \text{ hit}) = 0.01 \times 0.80 = 0.008$$

$$P(L1 \text{ miss} \wedge L2 \text{ miss}) = 0.01 \times 0.20 = 0.002$$

L1 Hit: 210 ns

L1 miss and L2 hit: 240 ns

Miss both: 440 ns

$$\text{EAT} = 0.99 \cdot 210 + 0.008 \cdot 240 + 0.002 \cdot 440 \approx 210.7 \text{ ns}$$

(We assume to read page table and data access are both 200 ns, so together 400 ns)

Because L1 TLB hits 99% of the time, almost every access avoids a page table search and only adds a TLB lookup latency of 10 ns on top of the memory access time.

If a rare L1 miss happens, L2 will most probably hit and that is just a 30 ns extra time delay.

Hence, EAT is 210.7 ns, only 10.7 ns worse than raw memory access. The average stays extremely low due to the rarity of L1 misses and the rarity that both L1 and L2 missing.

This shows how important how high TLB rates are.

---

3. Page Table Bits

Page size = 4 KB = 4096 = 2^{12} bytes.

Page Table Entry (PTE) size = 4 B

d = Page offset bits = $\log_2(4096) = \log_2(2^{12}) = 12$ bits

Entries/page = Page size ÷ PTE = 4096 ÷ 4 = 1024 = 2^{10}

(1)

32 bit total

d=12

p2=10

p1=10

Answer: d=12; p2=10; p1=10

(2)

64 bit total

d=12

p3=10

p2=10

p1=(64 - (12+10+10)) = 32

Answer: d=12; p3=10; p2=10; p1=32

Question 4: Address Translation

Assume we are now on a 12-bit system and use page size of 256 bytes. The OS maintains a list of free frames that are to be allocated in the order of {9, F, D}. A page table like below is already populated.

Page	Page Frame
0	0x4
1	0xB
2	0xA
3	-
4	-
5	0x2
6	-
7	0x0
8	0xC
9	0x1

A dash for a Page Frame indicates that the page is not in memory, i.e., needs to be allocated from the list of free frames. In the case of a page fault, you must use one of the free frames to update the page table.

1. Convert the virtual addresses {0x2A1} to its equivalent physical addresses in hexadecimal. Briefly illustrate your process of translation.
2. Convert the virtual addresses {0x4E6} to its equivalent physical addresses in hexadecimal. Briefly illustrate your process of translation.
3. Convert the virtual addresses {0x94A} to its equivalent physical addresses in hexadecimal. Briefly illustrate your process of translation.

Answer: Virtual address space: 12 bits -> maximum Virtual Address = 0xFFF

Note PFN => "Physical Frame Number"

1.

0x2A1 = 673

Page size = 256

Page number = floor(673 ÷ 256) = 2

Page offset = 673 mod 256 = 161 = 0xA1

FROM PAGE TABLE:

Page 2 gives frame 0xA, meaning the frame is present.

Physical address = PFN × frame size + page offset = 0xA × 256 + 161 = 2721 = 0xAA1

Answer = 0xAA1

2.

0x4E6 = 1254

Page size = 256

Page number = floor(1254 ÷ 256) = 4

Page offset = 1254 mod 256 = 230 = 0xE6

FROM PAGE TABLE:

Page 4 = not present -> page fault -> allocate next free frame, 0x9

Physical address = 0x9 × 256 + 230 = 2534 = 0x9E6

Answer = 0x9E6

3.

0x94A = 2378

Page size = 256

Page number = floor(2378 ÷ 256) = 9

Page offset = 2378 mod 256 = 74 = 0x4A

FROM PAGE TABLE:

Page 9 gives frame 0x1 (present)

Physical address = 0x1 × 256 + 74 = 330 = 0x14A

Answer = 0x14A

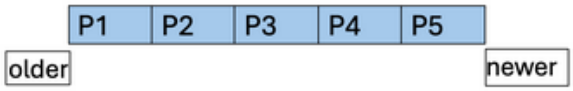
Question 5: Demand Paging

Assume there are 5 frames in physical memory. For each of the following reference strings:

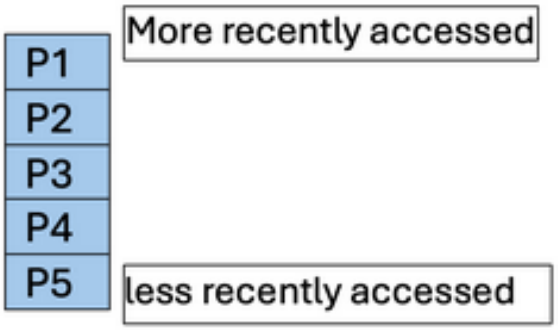
- {0,6,3,0,2,6,3,5,2,4,1,3,0,6,1,4,2,3,5,7}
- {3,1,4,2,5,4,1,3,5,2,0,1,1,0,2,3,4,5,0,1}
- {4,2,1,7,9,8,3,5,2,6,8,1,0,7,2,4,1,3,5,8}

1. Explain in your own word what Demand Paging means in OS.

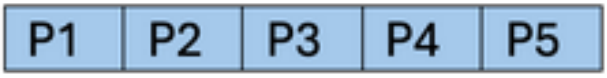
2. Assume you use a FIFO queue as illustrated in the diagram below to represent the 5 frames. Indicate the page numbers remaining in the physical memory at the end of the process. Indicate the number of page faults.



3. Use LRU page replacement, assume you use a stack below to represent the 5 frames. Indicate the page numbers remaining in the physical memory at the end of the process. Indicate the number of page faults.



4. Use OPT page replacement, assume you use a stack as illustrated in the diagram below to represent the 5 frames. Indicate the page numbers remaining in the physical memory at the end of the process. Indicate the number of page faults.



Answer: Demand Paging is only triggered when thereâĂŹs a demand. It only brings a page into physical memory when itâĂŹs actually needed, when itâĂŹs referenced. If the referenced page is not in RAM then it results in a page fault.

Reference String 1

$$\{0, 6, 3, 0, 2, 6, 3, 5, 2, 4, 1, 3, 0, 6, 1, 4, 2, 3, 5, 7\}$$

FIFO:

- Final frames: [3, 5, 7, 6, 2]
- Page faults: 13

LRU:

- Final frames: [4, 7, 2, 5, 3]
- Page faults: 13

OPT:

- Final frames: [7, 6, 3, 1, 4]
- Page faults: 10

Reference String 2

$$\{3, 1, 4, 2, 5, 4, 1, 3, 5, 2, 0, 1, 1, 0, 2, 3, 4, 5, 0, 1\}$$

FIFO:

- Final frames: [0, 3, 1, 2, 5]
- Page faults: 8

LRU:

- Final frames: [3, 5, 0, 1, 4]
- Page faults: 9

OPT:

- Final frames: [5, 1, 4, 2, 0]
- Page faults: 7

Reference String 3

$$\{4, 2, 1, 7, 9, 8, 3, 5, 2, 6, 8, 1, 0, 7, 2, 4, 1, 3, 5, 8\}$$

FIFO:

- Final frames: [5, 8, 7, 4, 3]

- Page faults: 17

LRU:

- Final frames: [4, 1, 3, 5, 8]
- Page faults: 18

OPT:

- Final frames: [5, 2, 1, 7, 8]
- Page faults: 13

Programming Section

Question 6: meminfo

This question investigates how Linux handles virtual and physical memory when a C program allocates memory using `malloc()`. The task involves allocating a user-specified number of pages based on the system page size (retrieved via `getpagesize()`), running the program with and without memory initialization, and then analyzing how the Resident Set Size (RSS) changes between the two cases.

In the implementation (available in the repository), `N` pages are allocated by multiplying the page count by the system page size. It also includes an optional flag to initialize the memory using `memset()`, which allows us to compare pure virtual allocation with actual physical allocation.

- **Without initialization:** only virtual memory is reserved; no physical memory is allocated.
- **With initialization:** each page is written to, triggering a minor page fault and causing the kernel to allocate a physical 4 KB page (demand paging).

To evaluate this behavior, the program was executed with:

```
/usr/bin/time --verbose ./my_allocator 128
```

```
/usr/bin/time --verbose ./my_allocator 128 1
```

The following table summarizes the measured memory behavior for 128 pages.

Metric	Without Initialization	With Initialization
Page Size	4096 bytes	4096 bytes
Number of Pages	128	128
Total Memory Allocated	524288 bytes (512 KB)	524288 bytes (512 KB)
Initialization	No	Yes (memset)
Address Returned by malloc()	0x71acbad7f010	0x7825abea1010
Maximum RSS	1600 KB	1984 KB
Minor Page Faults	77	204
Major Page Faults	0	0
Voluntary Context Switches	1	1
Involuntary Context Switches	0	0
User Time (s)	0.00	0.00
System Time (s)	0.00	0.00
Elapsed Time	0:00.00	0:00.00
CPU Usage	33%	100%
File System Inputs	0	0
File System Outputs	0	0
Command	./meminfo 128	./meminfo 128 1
Exit Status	0	0

Table 1: Side-by-Side Comparison of Memory Allocation With and Without Initialization

The RSS is higher in the initialization case because the memory is actually *touched*. Calling `malloc()` only reserves **virtual address space**, but the operating system does not allocate physical RAM until the memory is first accessed. This is known as **lazy allocation** or **demand paging**.

In the "no initialization" run, the program allocates 512 KB (128 pages), but never reads or writes to them. Since none of those pages are accessed, the kernel never backs them with physical frames. Therefore, the RSS remains close to the program's baseline size (around 1600 KB).

However, in the "with initialization" run, calling `memset()` writes to every allocated page. This forces the OS to allocate a physical page frame for each virtual page, increasing the RSS by approximately **512 KB**, which matches the difference observed:

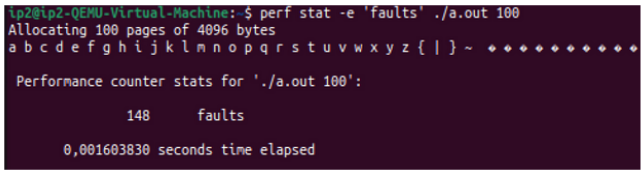
1984 KB - 1600 KB = **384 KB** (slightly less than 512 KB due to allocator metadata / alignment)

So the RSS differs because only the initialization run actually *touches* memory, causing physical pages to be committed.

Question 7: Page Faults

This is a programming assignment in C. You will create a simple C program that creates new mapping in the virtual address space for N pages backed by anonymous pages by using mmap(). N is set by the user.

For mmap(), you will experiment 2 options: (1) normal pages and (2) huge pages. You then will use the perf tool in Linux for profiling of page faults. Alternatively, use the same /usr/bin/time in question 6 to find out major and minor page faults. An example output is shown below.



1. You may modify the attached skeleton code ...
2. Run your program with Option 1, using N = 4096, 8192, 16384. Report the output in a screenshot.
3. Run your program with Option 2, using N = 4096, 8192, 16384. Report the output in a screenshot.
4. Explain the difference in the number of page faults and the printed elapsed time from using Options 1 and 2 for N=16384. Explain the observation in your own words.

Answer:
We investigated in this task how Linux handles memory allocation using `mmap()` and how page faults behave when accessing newly mapped pages. The program allocates **N pages**, writes once to each page (triggering demand paging), and compares two modes:

- **Option 1: Normal pages (4 KB)**
- **Option 2: Huge pages (2 MB, via MAP_HUGETLB)**

Measurements were collected using `/usr/bin/time -v`, focusing on minor page faults, major page faults, and execution time. The main goal was to observe how page fault counts scale with allocation size and how huge pages affect this behavior.

Results - Option 1 (Normal Pages)

Metric	4096 Pages	16384 Pages	81292 Pages
Page Size	4096 bytes	4096 bytes	4096 bytes
Allocation Type	Normal Pages	Normal Pages	Normal Pages
Allocation Success	Yes	Yes	Yes
Elapsed Time (s)	0.014028	0.047867	0.129395
Elapsed Cycles	14028	47867	129395
Maximum RSS (KB)	17896	67048	326760
Minor Page Faults	4172	16459	81366
Major Page Faults	0	0	0
User Time (s)	0.00	0.00	0.01
System Time (s)	0.01	0.05	0.14
Voluntary Context Switches	1	1	1
Involuntary Context Switches	0	1	1

Table 2: Execution Results Using Normal Pages (Option 1)

The execution time increases roughly linearly with the number of pages, reflecting the cost of touching more memory pages.

Results - Option 2 (Huge Pages)

Metric	4096 Pages	8192 Pages	16384 Pages
Page Size	4096 bytes	4096 bytes	4096 bytes
Allocation Type	Huge Pages	Huge Pages	Huge Pages
Allocation Success	Yes	Yes	Yes
Elapsed Time (s)	0.004814	0.009478	0.018088
Elapsed Cycles	4814	9478	18088
Characters Verified	a–p	a–p	a–p

Table 3: Execution Results Using Huge Pages (Option 2)

Execution time almost doubles when page count doubles. This is linear scaling, meaning the Cost == Number of pages touched.

Explanation of the difference in the number of page faults and the printed elapsed time from using Options 1 and 2 for N=16384.

With Normal Pages (Option 1):

1. We allocate 16,384 pages of 4 KB each = 64 MB total
2. The program writes to the first byte of each page in a loop
3. Each write to a new page triggers a minor page fault

4. Total faults = 16,384 + 75 baseline = 16,459 faults
5. Every single page requires kernel intervention to:
   (a) Allocate a physical page frame
   (b) Zero the page
   (c) Update the page table entry
   (d) Flush the TLB (Translation Lookaside Buffer)

With Huge Pages (Option 2):

1. We request the same 64 MB, but using 2 MB huge pages
2. 64 MB / 2 MB per huge page = only 32 huge pages needed
3. Each huge page covers 512 normal pages (2 MB / 4 KB = 512)
4. Total faults = 32 + 75 baseline = 107 faults
5. Much fewer kernel interventions needed!

The Math:
Normal pages: 16,384 data faults + 75 baseline = 16,459 total
Huge pages: 32 data faults + 75 baseline = 107 total

Reduction: 16,384 / 32 = 512x fewer data-related page faults!

Elapsed Time: 0.048s vs 0.016s

The huge page version is 3ÃŮ faster because:

1. Fewer Page Faults = Less Kernel Time
   (a) Each page fault requires switching from user mode to kernel mode
   (b) The kernel must allocate memory, update tables, and return
   (c) With 512x fewer faults, we spend 512x less time in fault handlers
2. Better TLB Efficiency
   (a) The TLB (Translation Lookaside Buffer) caches virtual–>physical address translations
   (b) With 4 KB pages: TLB can cache only a limited number of pages
   (c) With 2 MB huge pages: Each TLB entry covers 512ÃŮ more memory
   (d) Result: Fewer TLB misses during memory access
3. Less Memory Management Overhead

(a) Page tables are smaller (fewer entries needed)

(b) Less time updating and traversing page tables

(c) Better cache locality for page table entries

In simple terms, using huge pages is like telling the OS to give you a few large boxes to store your data instead of thousands of tiny envelopes. It's much quicker and more efficient for the OS to manage a few large boxes than it is to track thousands of individual envelopes.

Question 8: Page Reclamation

This is a programming assignment in C. You will create a simple C program that mimics the **active** and **inactive** lists in Linux kernel. The program initially generates a random page-reference string of 1000 accesses, where page numbers range from 0 to N. N is set by the user from argv[1]. The program maintains two linked list of **struct *Page***, for active and inactive lists, respectively. The struct *Page* has a reference bit, page id, and other auxiliary members.

Your program should create one pthread called ***player*** will play the generated reference string. For each page in the reference string, the player will set the reference bit to 1, move the page to the **rear** end of the active list, and sleep for 10 microseconds. When the size of active list is over 70% N, it will move about 20% pages at the **front** end of the active list to inactive list.

Another pthread called ***checker*** wakes up periodically after sleeping for M microseconds. M is set by the user from argv[2]. When it is up, it goes through all pages in the active list and reset their reference bit to 0. It also collects statistics of page references: for each page, it accumulates the number of times its reference bit is 1.

The program reports the final page statistics collected by checker in a screenshot. Report the pages in active and inactive list, respectively, in screenshot. An example output should look like this

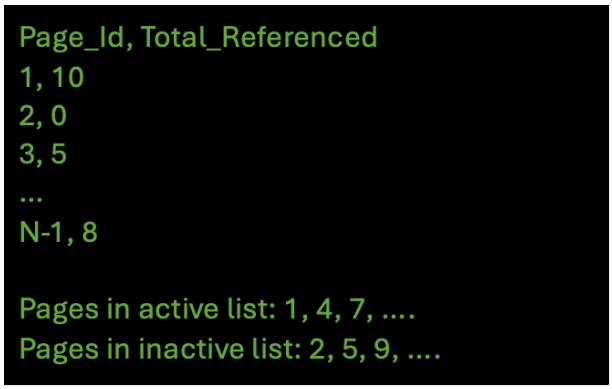


Figure 1: Enter Caption

1. You may use the skeleton code here. Feel free to modify the code to suit your purpose.
2. Describe your design.
3. Run your program with N: 64, M: 20. Report your output in screenshot.
4. Run your program with N: 64, M: 200. Report your output in screenshot.
5. Run your program with N: 64, M: 2000. Report your output in screenshot.
6. Explain in your own words the observation from 3-5.

Answer:

This Task simulates how an operating system reclaims memory using a two-list (*active* and *inactive*) page-management strategy executed by two concurrent threads:

1. **Player Thread (the Program):**
   - Simulates a program accessing a sequence of random memory pages.
   - When a page is accessed, it sets its `reference_bit` to 1 and moves it into the `active_list`, marking it as recently used.
   - If the `active_list` grows beyond 70% of the total page capacity, the oldest pages are demoted from the `active_list` to the `inactive_list`.
2. **Checker Thread (the OS Daemon):**
   - Periodically wakes up after a configurable sleep interval M microseconds.
   - Scans the `active_list`, detects pages with `reference_bit == 1`, increments their usage counter, and resets their `reference_bit` to 0.

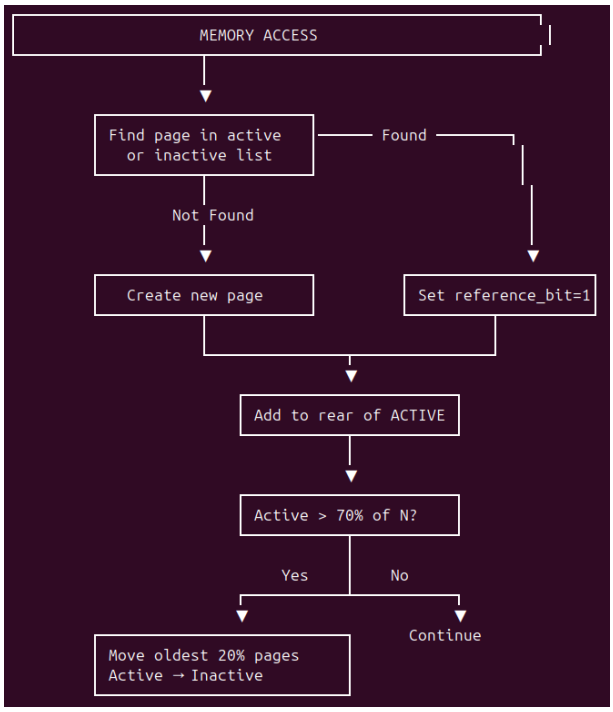


Figure 2: Flowchart of the Player Thread's Memory Access Logic

In summary, the Player continuously accesses pages and promotes them to an "active" state, while the Checker periodically inspects and clears this activity. The checking frequency M determines how precisely the system can distinguish "hot" pages from infrequently used ones. All shared data is synchronized using a mutex to prevent race conditions between the two threads.

Test Summary

Metric / Page ID	20 accesses	200 accesses	2000 accesses
Page 0	13	23	14
Page 1	15	12	14
Page 2	21	12	11
Page 3	11	13	12
Page 4	10	16	13
Page 5	10	18	11
Page 6	20	15	9
Page 7	20	14	11
Page 8	19	10	11
Page 9	11	16	11
Page 10	16	15	12
Page 11	11	17	10
⋮	⋮	⋮	⋮
Page 63	20	18	10
Active List	57,16,55,...,8	4,10,38,...,59	17,36,63,...,45
Inactive List	4,23,38,...,28	3,32,12,...,18	6,35,7,...,49

Table 4: Page Reclamation Results for 64 Pages at Different Access Counts

Test	N Pages	Checker Interval M	Active List	Inactive List	Max Ref.
1	64	20 μs (fast)	40 (62.5%)	24 (37.5%)	28
2	64	200 μs (medium)	38 (59.4%)	26 (40.6%)	24
3	64	2000 μs (slow)	33 (51.6%)	30 (46.9%)	18

Table 5: Summary of page reclamation behavior under different checker intervals.

Key Observations

1. Checker Frequency Controls Accuracy

A faster checker captures more reference events. As M increases, the system increasingly underestimates true page usage due to missed accesses.

2. Reference Bit is Lossy

Since the reference bit is binary, multiple accesses between two checker runs are counted as a single reference. Slower checks therefore compress the apparent difference between hot and cold pages.

3. Active/Inactive Distribution Shifts

- Fast checking (20 μs): Majority of pages stay in the active list.

- Slow checking (2000 μs): More pages are misclassified as "cold" and moved to the inactive list.

4. Active List Threshold Effects

Once the active list exceeds 70% of all pages, 20% of the oldest pages are demoted. With slow checking, this causes pages to be moved before their usage is detected.

Trade-Off Summary

Fast Checker (20 μs)

- + Accurate detection of hot pages
- - High CPU overhead

Slow Checker (2000 μs)

- + Low overhead
- - Misclassification of active pages; risk of thrashing

Medium Checker (200 μs) provides a balanced trade-off.

Conclusion

This experiment demonstrates the sensitivity of LRU-like page reclamation to sampling frequency. A 100× slower checker results in significantly lower reference counts and a shift toward inactive pages, showing how OS decisions depend heavily on how often memory usage is measured. Modern Linux systems address this using adaptive scanning rates based on memory pressure.